

เลขที่ห้องสอบ

มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าธนบุรี

ข้อสอบกลางภาคการศึกษาที่ 2/2557

วันที่ 23 กุมภาพันธ์ 2558

เวลา 09.00 – 12.00 น.

วิชา CPE 334 Operating Systems

น.ศ. วศ.คอมพิวเตอร์ปีที่ 3 หลักสูตรปกติ

คำสั่ง

- ข้อสอบมี 2 ส่วน ส่วนละ 50 คะแนน รวมเป็น 100 คะแนน
 - ส่วนที่ 1 มี 3 ข้อใหญ่
 - ส่วนที่ 2 มี 3 ข้อใหญ่
- ให้ทำข้อสอบส่วนที่ 1 ลงในตัวข้อสอบที่เว้นช่องไว้ให้ ส่วนที่ 2 ทำลงในสมุดคำตอบ
- อนุญาตให้ใช้เครื่องคำนวณ รูนและยี่ห้อที่มหาวิทยาลัยกำหนดได้
- ไม่อนุญาตให้นำเอกสารใดๆเข้าห้องสอบ
- เขียนชื่อและ รหัสประจำตัว ลงในกระดาษคำตอบทุกแผ่น (และแผ่นนี้)

(รศ.ดร.ธีรณี อจลากุล)

ผู้ออกข้อสอบส่วนที่ 1

(อ.ราชวิรัช สโรชวิกสิต)

ผู้ออกข้อสอบส่วนที่ 2

ข้อสอบนี้ได้ผ่านการประเมินจากภาควิชาวิศวกรรมคอมพิวเตอร์แล้ว

ชื่อ รหัสประจำตัว ภาควิชา/ชั้นปี

ส่วนที่ 1

I. (15 points) Mark all the correct choices in each question. Points will only be awarded when all the correct choices are selected.

1. The interval from the time of submission of a process to the time of completion is referred to as

- a) Waiting time
☒ b) Turnaround time
 c) Response time
 d) Throughput

Lec 3

- The **turnaround time** is defined as the time at which the job completes minus the time at which the job arrived in the system

$$T_{\text{turnaround}} = T_{\text{completion}} - T_{\text{arrival}}$$

2. Which statement(s) is/are true in priority scheduling

- ☒ a) CPU is allocated to the process with the highest priority
 b) CPU is allocated to the process with the ~~lowest~~ priority
 c) Equal priority processes cannot ~~not~~ be scheduled if a single queue is used *with other method*
☒ d) Priority scheduling policy can be used with the multiple queues concept *MLFQ*
 e) Round robin is the ~~only~~ policy that is used to schedule processes of the same priority

Lec 3

The Final Rules

- Rule 1:** If Priority(A) > Priority(B), A runs (B doesn't)
- Rule 2:** If Priority(A) = Priority(B), A & B run in RR
- Rule 3:** When a job enters the system, it is placed at the highest priority (the topmost queue)
- Rule 4:** Once a job uses up its time allotment at a given level (regardless of how many times it has given up the CPU), its priority is reduced (i.e. it moves down one queue)
- Rule 5:** After some time period S, move all the jobs in the system to the topmost queue

3. Processes are classified into different groups in

- a) The shortest job scheduling algorithm
 b) The round robin scheduling algorithm
 c) The shortest time to completion scheduling algorithm
☒ d) The multilevel queue scheduling algorithm

Lec 3

The Multi-Level Feedback Queue (MLFQ)

- Two goals
 - Optimize turnaround time
 - Minimize response time to allow a system to be responsive to interactive users
- MLFQ: Basic concept
 - Implement a number of **distinct queues of different priority levels**
 - At any given time, a job that is ready to run is on a single queue
 - MLFQ uses priorities to decide which queue should be selected
 - RR is used among jobs in the same queue
- The key to MLFQ scheduling lies in how the scheduler **sets queue priorities**

4. In an operating system, each process has its own

- a) User address space and global variables
☒ b) Heap and stack
 c) pending alarms, signals and signal handlers
 d) Ready queue

Lec 2

A Process Space

- A memory that the process uses to reference its instructions and data
 - Code Segment containing instructions
 - Data Segment containing **heap data** (malloc)
 - Stack** containing parameters, local variables, return address, etc. of functions
 - Process Descriptor (keep **Process Control Block**) – stored in the kernel space and can only be accessed in **kernel mode** execution



5. The address of the next instruction to be executed by the current process is provided by

a) CPU general purposed registers

☒ b) Program counter

c) Process stack

d) Process pipe

e) Process context

f) Process heap

6. Which of the following system calls does not return control to the calling point, on termination

Ref: Operating System Three Easy Pieces
Topic - Interlude: API (Page 2)

Now the interesting part begins. The process calls the `fork()` system call, which the OS provides as a way to create a new process. The odd part: the process that is created is an (almost) *exact copy* of the calling process. That means that to the OS, it now looks like there are two copies of the program `p1` running, and **both are about to return from the `fork()`**

a) fork

☒ b) exec

c) wait

d) none of the above

Lec 2

exec(): Family of Functions

- There is a family of **exec()** system calls, e.g. **execvp()**
- These are useful when you want to run a child program that is different from the parent program
- What it does: given the name of an executable (e.g. `word_count`), and some arguments (e.g. `p.txt`)

```
// child (new process)
char *myargs[3];
myargs[0] = strdup("word_count"); // program: "word_count"
myargs[1] = strdup("p.txt"); // argument: file to count
myargs[2] = NULL; // marks end of array
execvp(myargs[0], myargs); // runs the word count code
printf("this shouldn't print out");
```

- A successful call to an **exec()** function never returns

Lec 2

wait(): Example Usage

- Sometimes, it is quite useful for a parent to wait for a child process to finish before moving on
- Using **wait()** or **waitpid()** will force the parent to wait
- Let's modify the last `printf()` of the previous code:

```
// parent goes down this path (original process)
{
    int wc = wait(NULL);
    printf("hello, I am parent of %d (wc:%d) (pid:%d)\n",
           rc, wc, (int) getpid());
}
```

- The parent calls **wait()** to delay its execution until the child finishes executing - **when the child is done, wait() returns to the parent**

7. The following C program :

main()

```
{
    fork();
    fork();
    printf("hello");
}
```

How many times the word "hello" will be printed

a) Only once

b) Twice

c) Three times

☒ d) Four times

e) Eight times

Ref: Operating System Three Easy Pieces
Topic - Interlude: API (Page 3)

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <string.h>
5 #include <sys/wait.h>
6
7 int
8 main(int argc, char *argv[])
9 {
10     printf("hello world (pid:%d)\n", (int) getpid());
11     int rc = fork();
12     if (rc < 0) { // fork failed; exit
13         fprintf(stderr, "fork failed\n");
14         exit(1);
15     } else if (rc == 0) { // child (new process)
16         printf("hello, I am child (pid:%d)\n", (int) getpid());
17         char *myargs[3];
18         myargs[0] = strdup("wc"); // program: "wc" (word count)
19         myargs[1] = strdup("p3.c"); // argument: file to count
20         myargs[2] = NULL; // marks end of array
21         execvp(myargs[0], myargs); // runs word count
22         printf("this shouldn't print out");
23     } else { // parent goes down this path (main)
24         int wc = wait(NULL);
25         printf("hello, I am parent of %d (wc:%d) (pid:%d)\n",
26                rc, wc, (int) getpid());
27     }
28     return 0;
29 }
```

Figure 5.3: Calling `fork()`, `wait()`, And `exec()` (p3.c)

The `fork()` system call is strange; its partner in crime, `exec()`, is not so normal either. What it does: given the name of an executable (e.g., `wc`), and some arguments (e.g., `p3.c`), it loads code (and static data) from that executable and overwrites its current code segment (and current static data) with it; the heap and stack and other parts of the memory space of the program are re-initialized. Then the OS simply runs that program, passing in any arguments as the `argv` of that process. Thus, it does *not* create a new process; rather, it transforms the currently running program (formerly `p3`) into a different running program (`wc`). After the `exec()` in the child, it is almost as if `p3.c` never ran; a successful call to `exec()` never returns.

8. If a thread invokes the `exec` system call, Which statement(s) is/are true

a) Only the `exec` executes as a separate process

☒ b) The program specified in the parameter to `exec` will replace the entire process

c) The `exec` is ignored as it is invoked by a thread

d) None of the above is correct

9. A process having multiple threads of control implies that ...

- ☒ a) It can do more than one task at a time
- b) It can do only one task at a time, but much faster
- c) It has to use only one thread per process
- d) None of the above is correct

Lee7

Introduction

- We now move on to the second of the main themes of this course – **Concurrency**
- Concurrency exists at the OS level between different processes executing at the same time
- Concurrency can also exist within a single process
 - Here we introduce a new abstraction for concurrency within a single process: the **thread**
 - A **multi-threaded program or process has more than one point of execution**
- Nowadays, with multi-core CPUs, different processes or threads may execute in parallel on different cores

10. Resource sharing helps

- a) Share the memory and resources of the process to which the threads belong
- ☒ b) Applications to have several different threads of activities all within the same address space
- c) Reduce the address space that a process could potentially use
- d) Reduce the time needed to create a process

Lee7

Threads

- Each thread is very much like a separate process, except that they share the same address space and thus can access the same data
- A thread has
 - A **program counter (PC)** that is used to fetch the next instruction to be executed by that thread
 - A private set of registers for computation
- Thus, if there are two threads on a single processor, a threads' **context-switch** must take place
- Even on a multi-core CPU, there are generally many more threads than there are cores

11. Which of the following is FALSE

- a) Context switch time is longer for kernel level threads than for user level threads
- b) User level threads do not need any hardware support
- c) Related kernel level threads can be scheduled on different processors in a multiprocessor system

OS 30000000: thread unblock

- ☒ d) Blocking one kernel level thread blocks all other related threads

Ref: Operating System Three Easy Pieces

Topic - The Abstraction: The Process (Page 5)

12. The state of a process is defined by

- a) The final activity of the process
- b) The activity just executed by the process
- c) The activity that will be executed next by the process
- ☒ d) The current activity of the process

Process States

Now that we have some idea of what a process is (though we will continue to refine this notion), and (roughly) how it is created, let us talk about the different **states a process can be in at a given time**. The notion that a process can be in one of these states arose in early computer systems [DV66,V+65]. In a simplified view, a process can be in one of three states:

- Running:** In the running state, a process is running on a processor. This means it is executing instructions.
- Ready:** In the ready state, a process is ready to run but for some reason the OS has chosen not to run it at this given moment.
- Blocked:** In the blocked state, a process has performed some kind of operation that makes it not ready to run until some other event takes place. A common example: when a process initiates an I/O request to a disk, it becomes blocked and thus some other process can use the processor.

13. Which of the following is not the state of a process?

- ☒ a) New
- ☒ b) Old
- ☒ c) Waiting
- d) Running
- e) Ready
- ☒ f) Terminated

Lee2

Process State

- Ready:** a process is ready to run *but in some reason OS has chosen not to run it at the given moment*
- Running:** a process is running on a processor *(is executing instructions)*
- Blocked:** a process has performed some operation that means it not ready to run, it is then suspended *it has to wait for an I/O*



14. In UNIX, each process is identified by its

- a) Process Control Block
- b) Device Queue
- ☒ c) Process Identifier
- d) None of these

Ref: Operating System Three Easy Pieces
Topic - Interlude: API (Page 2)

Let us understand what happened in more detail in p1.c. When it first started running, the process prints out a hello world message; included in that message is its **process identifier**, also known as a **PID**. The process has a PID of 29146; in UNIX systems, the PID is used to name the process if one wants to do something with the process, such as (for example) stop it from running. So far, so good.

15. If one thread opens a file with read privileges then

- a) Other threads in the another process can also read from that file
- ☒ b) Other threads that belong the same process can also read from that file
- c) All other threads cannot read from that file
- d) All of the mentioned

ชื่อรหัสประจำตัว.....ภาควิชา/ชั้นปี.....

II. (15 points) Explain how a commercial bank handles customers during rush hours with the concept of the Multi-level Feedback Queue (MLFQ), in your opinion. Describe the algorithms and policies used and provide suggestions on how to improve the process with the goal of optimizing the average turnaround time of customers.

III. (20 points)

- a) (10 points) Describe the concept of “context switching” and provide step-by-step information of how it is performed by the operating system.
- b) (10 points) What are the primary differences between calling a function and cooperatively switching to (and from) another thread at user level, in the same address space? Is one more expensive than the other? Why or why not?

ส่วนที่ 2

1. จากเนื้อหาที่เกี่ยวกับการจัดการหน่วยความจำหลัก (Main Memory) จงตอบคำถามต่อไปนี้ (20 คะแนน)

1.1. จงอธิบายการเกิดขึ้นของ Internal Fragmentation และ External Fragmentation สำหรับการจองพื้นที่ในหน่วยความจำหลักแบบใช้พื้นที่ต่อเนื่อง (Contiguous Memory Allocation) พร้อมทั้งอธิบายถึงผลกระทบและแนวทางการจัดการว่าเหมือนหรือแตกต่างกันอย่างไร

1.2. การจองพื้นที่ในหน่วยความจำหลัก โดยใช้แนวคิดที่เรียกว่า Paging Scheme จำเป็นต้องใช้ Page Table จงอธิบายถึงความสำคัญของ Page Table ส่วนการเพิ่มประสิทธิภาพการทำงานของ Page Table สามารถใช้เทคนิคที่เรียกว่า Inverted Page Table จงอธิบายหลักการทำงานของ Inverted Page Table และแง่มุมในการเพิ่มประสิทธิภาพ

1.3. จงคำนวณหา Physical Address จาก Logical Address 16 บิต เมื่อระบบปฏิบัติการใช้วิธี Segmentation เพื่อจองพื้นที่ในหน่วยความจำหลัก และมี Segment Table ดังตารางด้านล่าง โดยสมมติให้ 4 บิตบนเป็นตัวบอกเซกเมนต์ (ต้องตอบเป็นฐานสิบ)

Segment	Base	Size
Code	32K	2K
Heap	34K	2K
Stack	28K	2K

1.3.1. Logical Address เป็น 110 (ฐานสิบ)

1.3.2. Logical Address เป็น 2558 (ฐานสิบ)

1.3.3. Logical Address เป็น 12528 (ฐานสิบ)

2. จากเนื้อหาที่เกี่ยวกับการจัดการหน่วยความจำเสมือน (Virtual Memory) จงตอบคำถามต่อไปนี้ (20 คะแนน)

2.1. ถ้าเปรียบเทียบระบบปฏิบัติการ 2 ตัว ที่มีการเลือกใช้ขนาด Page แตกต่างกันคือ ขนาดเล็กกับขนาดใหญ่ จะส่งผลกับอัตราการเกิด Page Fault แตกต่างกันอย่างไรร

2.2. จงแสดงตาราง Frame ของหน่วยความจำหลัก และ จงคำนวณ Effective Access Time เมื่อมีสถานะการณ์การเข้าถึง Page ตามลำดับต่อไปนี้

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1 (นับได้ 20 ตัว)

สมมติให้

- จำนวน Frame คือ 4

- เวลาในการเข้าถึงหน่วยความจำหลักเป็น 9 ns
- เวลาให้การเข้าถึงหน่วยความจำสำรอง 25 ms

2.2.1 เมื่อใช้ Page Replacement Algorithm เป็นแบบ LRU

2.2.2 จากข้อมูลเวลาในการเข้าถึงหน่วยความจำหลักและสำรองตามที่กำหนดไว้ ถ้าต้องการให้ EAT ชี้ว่าเวลาในการเข้าถึงหน่วยความจำหลักไม่เกิน 100 เท่า page fault rate ควรมีความเป็นอย่างไร

3. ถ้ามีบริษัทเอกชนต่างชาติรายใหญ่ (นามไทยสมมุติว่า “เล็กนึม”) จ้างนักศึกษาเข้าทำงานและมอบหมายงานให้ปรับปรุงระบบปฏิบัติการชื่อดัง (นามไทยสมมุติว่า “หน้าต่าง”) ในส่วนการบริหารจัดการหน่วยความจำหลัก (Memory Management) โดยให้แนวคิดที่ต้องปรับปรุงเทคนิคการจองพื้นที่ในหน่วยความจำหลัก (Memory Allocation) จากแบบเดิมใช้ Paging Scheme พัฒนาให้เป็นแบบผสมระหว่าง Paging Scheme และ Segmentation Scheme นักศึกษาจงอธิบายการทำงานงานของเทคนิคการจองพื้นที่ในหน่วยความจำหลักแบบผสมนี้ โดยใช้ Diagram นำเสนอ พร้อมทั้งอธิบายโดยละเอียดของการทำ Address Binding (ลักษณะและลำดับการนำเสนอมีผลต่อการพิจารณาคะแนนด้วย) (10 คะแนน)